

Statistical Properties of Text and Index Compression

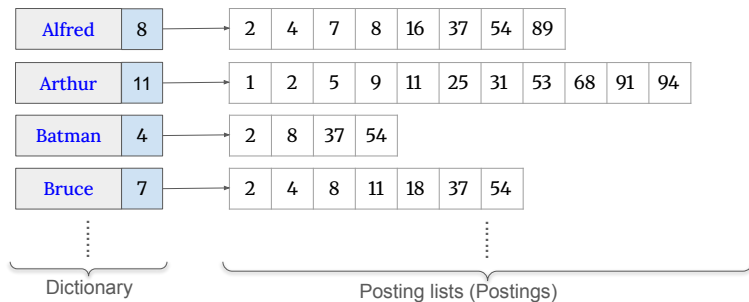
Introduction to Information Retrieval

Dwaipayan Roy

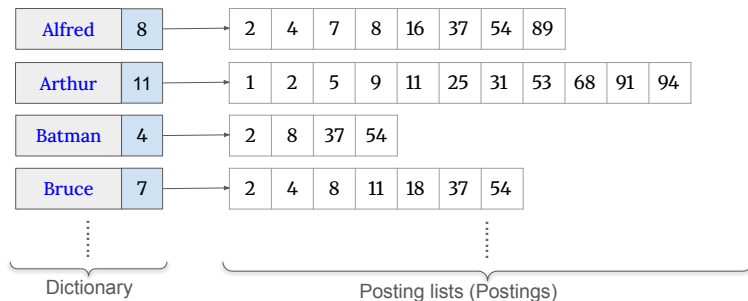
Department of Computational and Data Sciences



Today's agenda

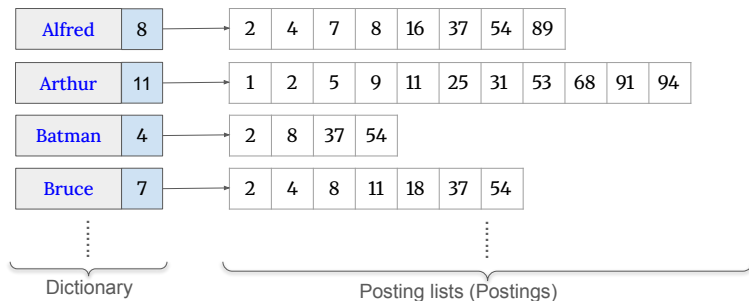


Today's agenda



- How can we **compress** the **dictionary** of the inverted index?
- How can we **compress** the **postings** of the inverted index?

Today's agenda



- How can we **compress** the **dictionary** of the inverted index?
- How can we **compress** the **postings** of the inverted index?
- **Term statistics**: how terms distributed in document collections?

Why compression? (in general)

- Use less disk space (**saves money**).
- Keep more stuff in memory (**increases speed**).
- Increase speed of transferring data from disk to memory (**increases speed**):
 - ▶ read compressed data and decompress in memory is **faster than** read uncompressed data.

Why compression in information retrieval?

- Dictionary compression:
 - ▶ Make it small enough to keep in main memory

Why compression in information retrieval?

- Dictionary compression:
 - ▶ Make it small enough to keep in main memory
- Postings compression:
 - ▶ Reduce disk space needed, decrease time needed to read from disk.
 - 📌 Large search engines keep significant part of postings in memory.

Lossy vs. lossless compression

Lossy vs. lossless compression

- Lossy compression: discard some information
 - ▶ downcasing, stopwords removal, stemming, number elimination etc.

Lossy vs. lossless compression

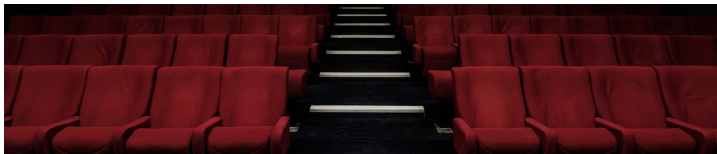
- Lossy compression: discard some information
 - ▶ downcasing, stopwords removal, stemming, number elimination etc.
- Lossless compression: all information preserved.
 - ▶ mostly done in index compression.

Statistical Properties of Text

- There are many ways to convey the same information (which makes IR difficult).
- Are there any statistical properties of word usage that are **predictable**?
 - ▶ across languages? across modalities? across genres?

Statistical Properties of Text

Wikipedia movie dataset



['Release Year', 'Title', 'Origin/Ethnicity', 'Director', 'Cast', 'Genre', 'Wiki Page', 'Plot']

- Number of documents: 34,886
- Number of terms (vocabulary size): 12,893,736
- Number of unique terms : 132,004

Dataset source: <https://www.kaggle.com/datasets/jrobischon/wikipedia-movie-plots>

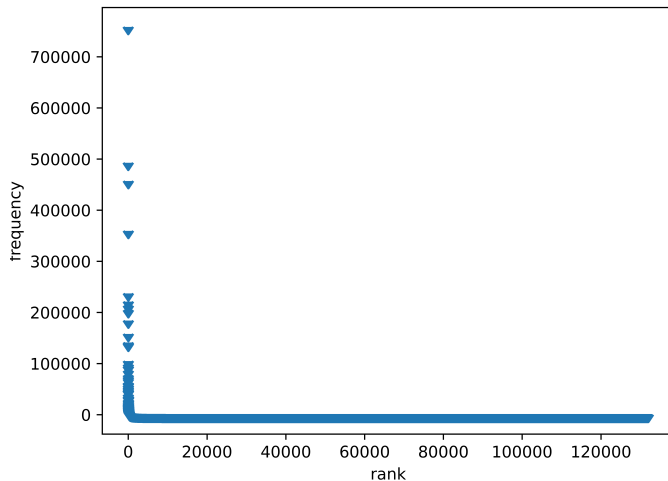
Statistical Properties of Text

Wikipedia movie dataset - top 20 most frequent words

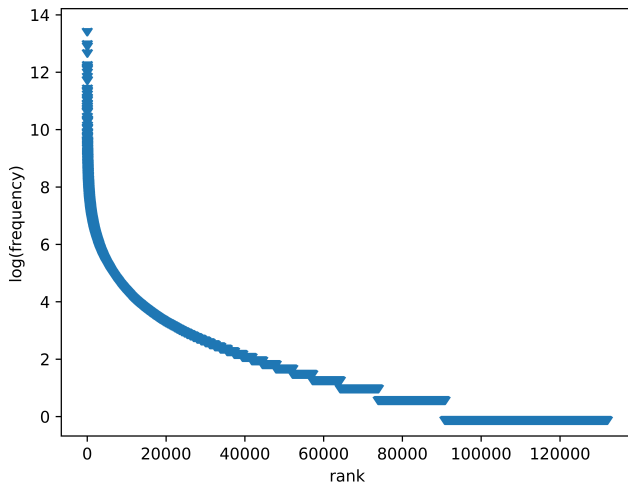
Rank	Term	Frequency	Rank	Term	Frequency
1	the	758427	11	that	141264
2	to	492810	12	with	138881
3	and	457159	13	him	104913
4	a	359491	14	for	97990
5	of	237230	15	she	94305
6	is	221227	16	by	93979
7	in	213134	17	as	85736
8	his	204553	18	on	76887
9	he	184269	19	but	73411
10	her	157961	20	who	72884

Wikipedia Movie Dataset

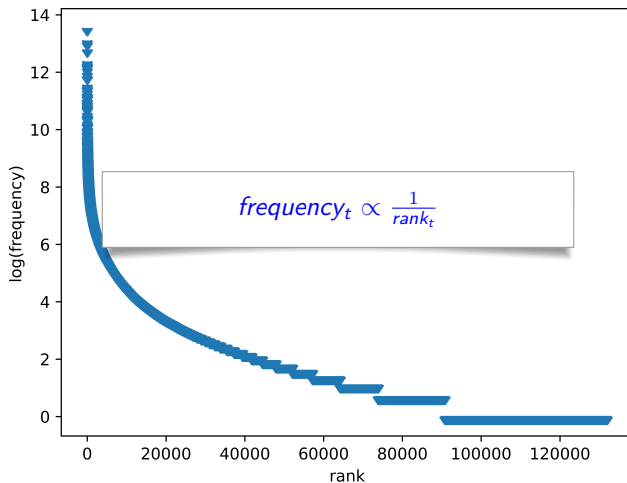
frequency versus rank distribution



Wikipedia Movie Dataset



Wikipedia Movie Dataset



Zipf's Law

- Term-frequency decreases **rapidly** as a function of rank

Zipf's Law

$$f_t = \frac{k}{r_t}$$

- f_t = frequency (number of times term t occurs)
- r_t = frequency-based rank of term t
- k = constant (specific to the collection)

Zipf's Law

Zipf's Law

$$\frac{1}{N} * f_t = \frac{1}{N} * \frac{k}{r_t}$$

Zipf's Law

Zipf's Law

$$\frac{1}{N} * f_t = \frac{1}{N} * \frac{k}{r_t}$$

$$P(t) = \frac{c}{r_t}$$

- N = total number of terms in the collection
- c = constant (for English $c = 0.1$)

Zipf's Law

$$P(t) = \frac{c}{r_t} (c = 0.1)$$

- The most frequent term accounts for 10% of the text
- The second most frequent term accounts for 5%
- The third most frequent term accounts for about 3%
- Together, the top 10 account for about 30%
- Together, the top 20 account for about 36%
- Together, the top 50 account for about 45%

Zipf's Law

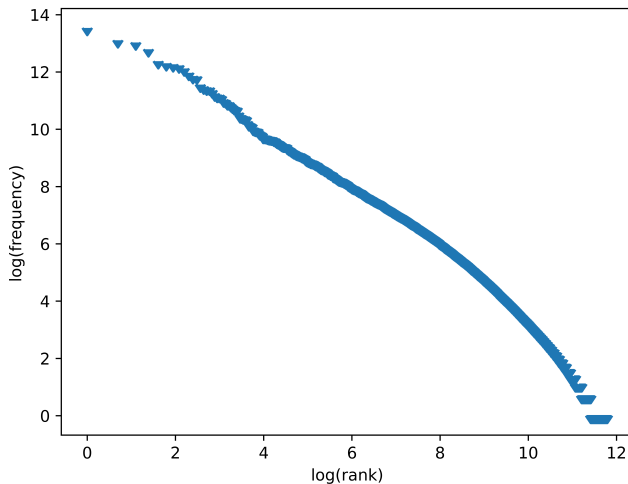
Visualization

$$f_t = \frac{k}{r_t}$$

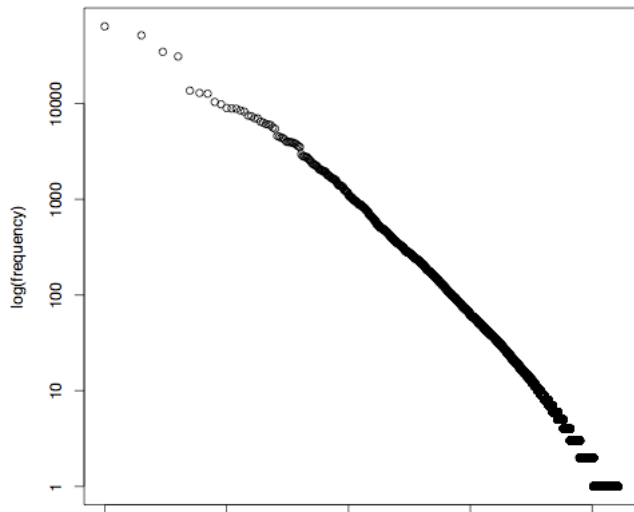
$$\log(f_t) = \log(k) - \log(r_t)$$

- If Zipf's law holds true $\rightarrow \log(f_t)$ vs. $\log(r_t)$ plot will be a straight line with a slope of -1.

Wikipedia movie dataset: Zipf's Law



Bible: Zipf's Law



Zip's Law

- Zipf's Law holds true for:
 - ▶ different sizes of text
 - ▶ different genres
 - ▶ different topics
 - ▶ different complexity of content
 - ▶ different languages

Zipf's Law

implications

in the dark knight gotham city is plagued by rising crime and the mysterious vigilante batman is determined to restore order batmans efforts are complicated by the arrival of the joker a sadistic criminal mastermind who thrives on chaos and seeks to dismantle the citys moral fabric as the joker escalates his attacks he forces batman into a moral dilemma pushing him to confront his own ethics and the thin line between justice and revenge meanwhile the joker manipulates key figures like harvey dent turning him into the vengeful vigilante two-face in the end batman takes the fall for harveys crimes becoming a fugitive to protect gothams hope leaving the city to believe in the myth of batman as a symbol of heroism

- The most descriptive words → do not appear in every document.
- Zipf's law allows us to automatically identify these non-descriptive terms and treat them differently

Zipf's Law

implications

in the dark knight gotham city is plagued by rising crime and the mysterious vigilante batman is determined to restore order batmans efforts are complicated by the arrival of the joker a sadistic criminal mastermind who thrives on chaos and seeks to dismantle the citys moral fabric as the joker escalates his attacks he forces batman into a moral dilemma pushing him to confront his own ethics and the thin line between justice and revenge meanwhile the joker manipulates key figures like harvey dent turning him into the vengeful vigilante two-face in the end batman takes the fall for harveys crimes becoming a fugitive to protect gothams hope leaving the city to believe in the myth of batman as a symbol of heroism

- Ignoring the most frequent terms greatly reduces the size of the index.
 - ▶ These have very long inverted lists.

Zipf's Law

implications

in the dark knight gotham city is plagued by rising crime and the mysterious vigilante batman is determined to restore order batmans efforts are complicated by the arrival of the joker a sadistic criminal mastermind who thrives on chaos and seeks to dismantle the citys moral fabric as the joker escalates his attacks he forces batman into a moral dilemma pushing him to confront his own ethics and the thin line between justice and revenge meanwhile the joker manipulates key figures like harvey dent turning him into the vengeful vigilante two-face in the end batman takes the fall for harveys crimes becoming a fugitive to protect gothams hope leaving the city to believe in the myth of batman as a symbol of heroism

- Ignoring the most frequent terms greatly reduces the size of the index.
 - ▶ These have very long inverted lists.
 - ▶ These words can be important in combination with others (e.g., in proximity operators).

Zipf's Law

implications

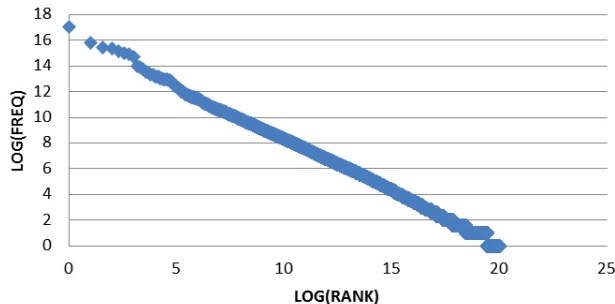
in the dark knight gotham city is plagued by rising crime and the mysterious vigilante batman is determined to restore order batmans efforts are complicated by the arrival of the joker a sadistic criminal mastermind who thrives on chaos and seeks to dismantle the citys moral fabric as the joker escalates his attacks he forces batman into a moral dilemma pushing him to confront his own ethics and the thin line between justice and revenge meanwhile the joker manipulates key figures like harvey dent turning him into the vengeful vigilante two-face in the end batman takes the fall for harveys crimes becoming a fugitive to protect gothams hope leaving the city to believe in the myth of batman as a symbol of heroism

- Ignoring the most frequent terms greatly reduces the size of the index.
 - ▶ These have very long inverted lists.
 - ▶ These words can be important in combination with others (e.g., in proximity operators).
- **Alternative:** leave them in index; remove them from query, unless they occur in proximity operator.

Zipf's Law

for queries

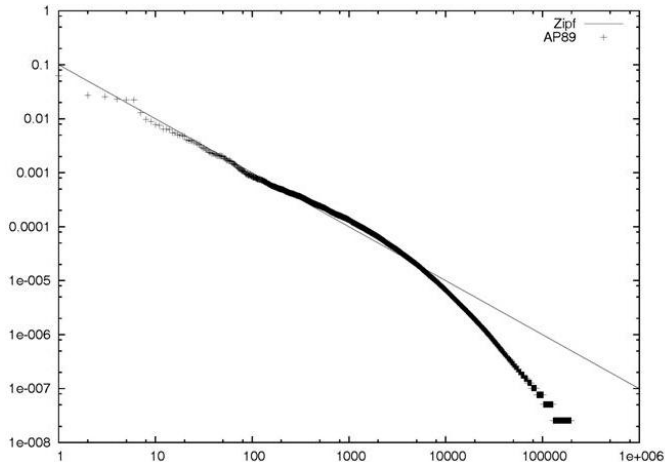
AOL Query Log



- Same trend: a few queries occur very frequently, while most occur very infrequently
- **Opportunity**: system can be tweaked to do well on those queries it is likely to [see](#) frequently.

Zipf's Law

rare terms



- Poor fit for rare terms.

Zipf's Law

implication

- Given Zipf's Law, as a collection grows, how will the size of the vocabulary grow?

Heaps' Law

- The number of new words decreases as the size of the corpus increases

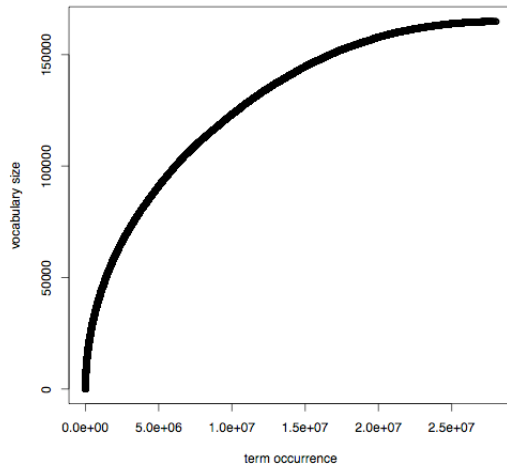
Heaps' Law

$$v = k * n^{\beta}$$

- v = size of the vocabulary (number of unique words)
- n = size of the corpus (number of word-occurrences)
- k = constant ($10 \leq k \leq 100$) (not the same as k in Zipfs law)
- β = constant ($\beta \approx 0.50$)

Heaps' Law

IMDB Corpus



Heaps' Law

Implications

- As corpus grow, number of new terms increase dramatically at first, then increase at a slower rate.

Heaps' Law

Implications

- As corpus grow, number of new terms increase dramatically at first, then increase at a slower rate.
- Nevertheless, as corpus grow, new terms will always be found (even if the corpus becomes huge).
why?

Heaps' Law

Implications

- As corpus grow, number of new terms increase dramatically at first, then increase at a slower rate.
- Nevertheless, as corpus grow, new terms will always be found (even if the corpus becomes huge).

why?

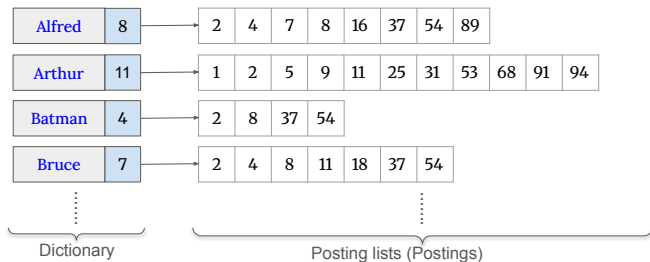
- ▶ there is no end to vocabulary growth
 - ★ invented words, proper nouns (people, products), misspellings, email addresses, etc.

Heaps' Law

Implications

- Given a corpus and a new set of data, the number of new index terms will depend on the size of the corpus
- Given more data, new index terms will always be required
- Also be true for controlled vocabularies
 - ▶ Given an existing corpus and a new dataset, requirement for new concepts will depend on the size of the corpus.
 - ▶ Given more data, new concepts will always be observed.

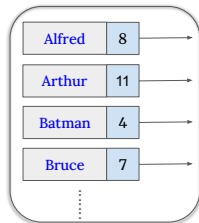
Dictionary compression



- Dictionary → containing all term in the collection.
 - ▶ small compared to the postings file.
- Needed to be kept in the memory.
- **Challenges:** other applications taking up memories, limited memory etc.
- So compressing the dictionary is important.

Dictionary

as array of fixed-width entries

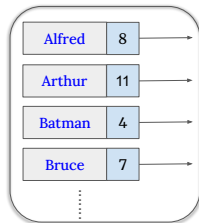


Term (String)	Document frequency (int)	Pointer to postings list (pointer)
------------------	-----------------------------	---------------------------------------

- `sizeof(char)` - 1 byte
- `sizeof(int)` - 4 bytes
- `sizeof(pointer)` - 4 bytes
- space needed:
 - ▶ term - 20/80 bytes
 - ▶ frequency - 4 bytes
 - ▶ pointer- 4 bytes

Dictionary

as array of fixed-width entries



Term (String)	Document frequency (int)	Pointer to postings list (pointer)
------------------	-----------------------------	---------------------------------------

- `sizeof(char)` - 1 byte
- `sizeof(int)` - 4 bytes
- `sizeof(pointer)` - 4 bytes
- space needed:
 - ▶ term - 20/80 bytes
 - ▶ frequency - 4 bytes
 - ▶ pointer- 4 bytes
- Needed to be stored for each item in the dictionary.
- Total space requirement = $28 \text{ (or, } 88) * (\# \text{ unique terms}) \text{ bytes.}$

Dictionary

as array of fixed-width entries: problem

- Most of the bytes in the term column are wasted.
- Can't handle terms with length >20 characters
 - ▶ floccinaucinihilipilification
 - ▶ hydrochlorofluorocarbons
 - ▶ supercalifragilisticexpialidocious

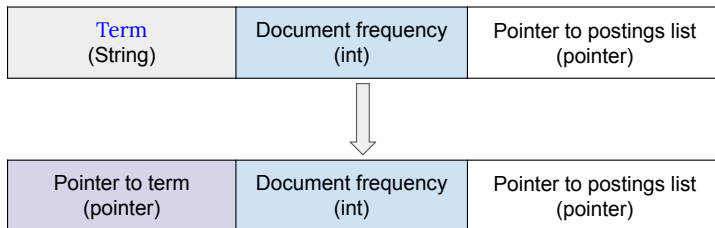
Dictionary

as array of fixed-width entries: problem

- Most of the bytes in the term column are wasted.
- Can't handle terms with length >20 characters
 - ▶ floccinaucinihilipilification
 - ▶ hydrochlorofluorocarbons
 - ▶ supercalifragilisticexpialidocious
- Average length of a term in English: 8 characters.
- Source of wastage: variable length of strings.
- How can we use on average 8 characters per term?

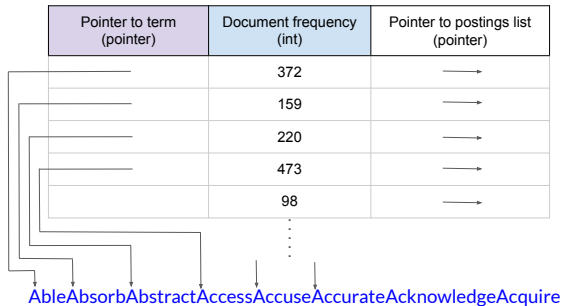
Solution

dictionary as a string



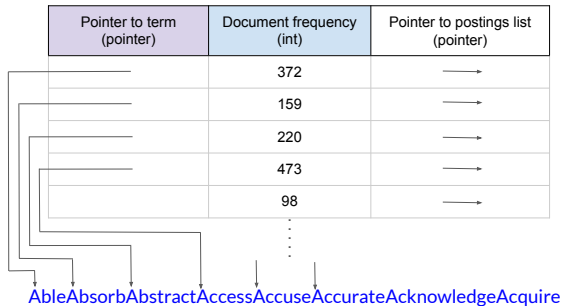
Solution

dictionary as a string



Solution

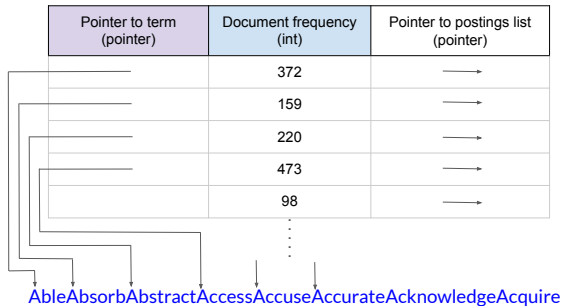
dictionary as a string



- 4 bytes per term for frequency
- 4 bytes per term for pointer to postings list
- 8 bytes (on average) for term in string

Solution

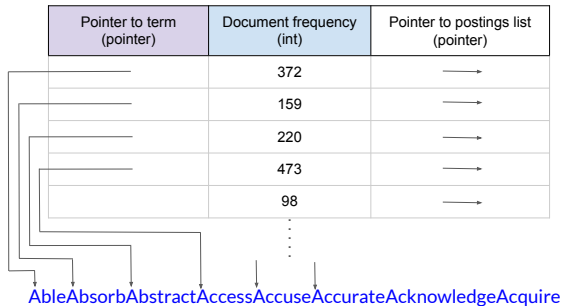
dictionary as a string



- 4 bytes per term for frequency
- 4 bytes per term for pointer to postings list
- 8 bytes (on average) for term in string
- 3 bytes per pointer into string
 - ▶ need to resolve $400000 * 8 = 3.2 * 10^6$ positions;
 - ▶ they need to be $\log_2 3.2 * 10^6 \approx 22$ bits or 3 bytes long.

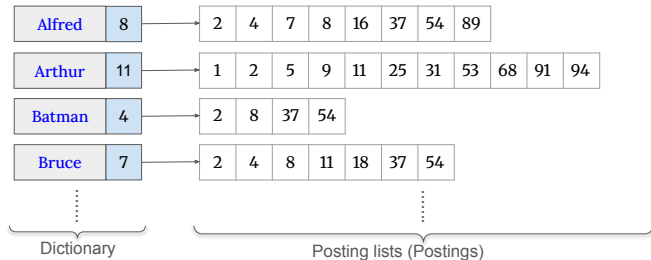
Solution

dictionary as a string

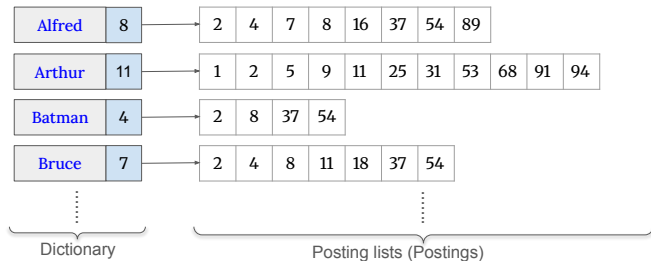


- 4 bytes per term for frequency
- 4 bytes per term for pointer to postings list
- 8 bytes (on average) for term in string
- 3 bytes per pointer into string
 - ▶ need to resolve $400000 * 8 = 3.2 * 10^6$ positions;
 - ▶ they need to be $\log_2 3.2 * 10^6 \approx 22$ bits or 3 bytes long.
- Space: $400,000 * (4+4+3+8)$

Postings compression

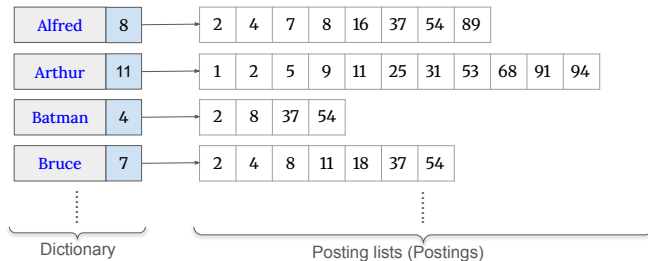


Postings compression



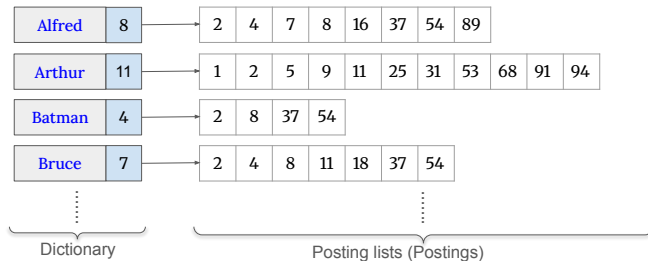
- The postings file is much larger than the dictionary

Postings compression



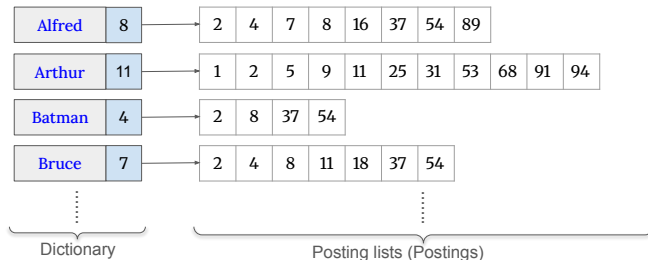
- The postings file is much larger than the dictionary - factor of at least 10 or more.

Postings compression



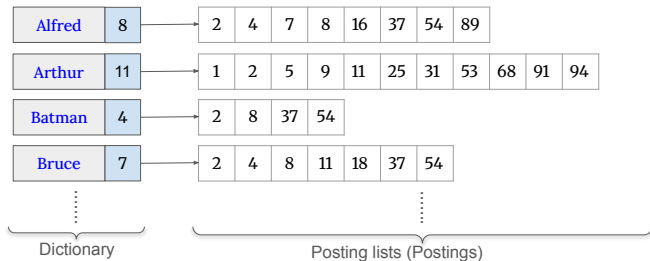
- The postings file is much larger than the dictionary - factor of at least 10 or more.
- **Target:** store each posting values *compactly*.

Postings compression



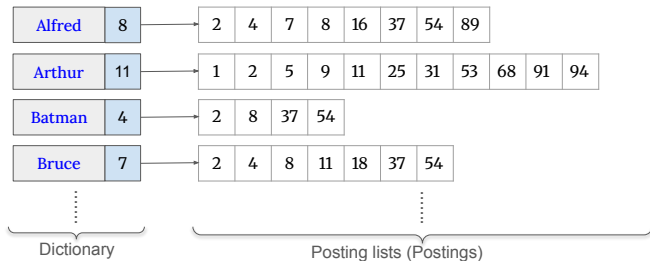
- The postings file is much larger than the dictionary - factor of at least 10 or more.
- **Target:** store each posting values *compactly*.
- A posting value → document identifier (docid).

Postings compression



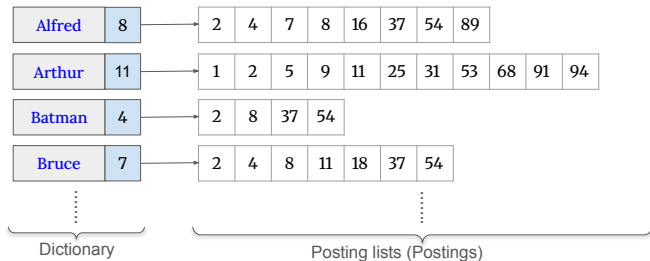
- The postings file is much larger than the dictionary - factor of at least 10 or more.
- **Target:** store each posting values *compactly*.
- A posting value → document identifier (docid).
- Number of documents in RCV1 → 800,000 documents

Postings compression



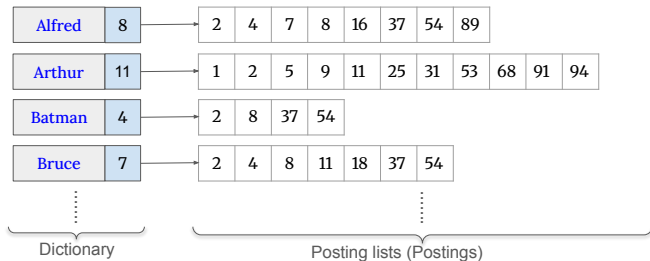
- The postings file is much larger than the dictionary - factor of at least 10 or more.
- **Target:** store each posting values *compactly*.
- A posting value $\rightarrow$ document identifier (docid).
- Number of documents in RCV1 $\rightarrow$ 800,000 documents $\rightarrow$ uses 32 bits/docid with 4-byte integers.

Postings compression



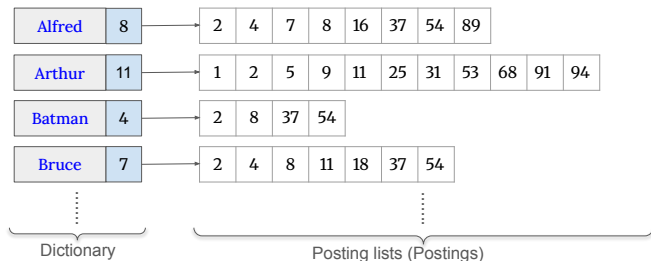
- The postings file is much larger than the dictionary - factor of at least 10 or more.
- **Target:** store each posting values *compactly*.
- A posting value $\rightarrow$ document identifier (docid).
- Number of documents in RCV1 $\rightarrow$ 800,000 documents $\rightarrow$ uses 32 bits/docid with 4-byte integers.
- Essentially we need not more than -

Postings compression



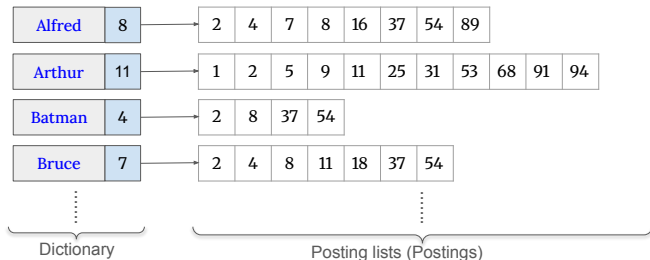
- The postings file is much larger than the dictionary - factor of at least 10 or more.
- **Target:** store each posting values *compactly*.
- A posting value $\rightarrow$ document identifier (docid).
- Number of documents in RCV1 $\rightarrow$ 800,000 documents $\rightarrow$ uses 32 bits/docid with 4-byte integers.
- Essentially we need not more than - **20 bits** ($\log_2 800,000 \approx < 20$) per docid.

Postings compression



- The postings file is much larger than the dictionary - factor of at least 10 or more.
- **Target:** store each posting values *compactly*.
- A posting value $\rightarrow$ document identifier (docid).
- Number of documents in RCV1 $\rightarrow$ 800,000 documents $\rightarrow$ uses 32 bits/docid with 4-byte integers.
- Essentially we need not more than - 20 bits ($\log_2 800,000 \approx < 20$) per docid. **Why?**

Postings compression



- The postings file is much larger than the dictionary - factor of at least 10 or more.
- **Target:** store each posting values *compactly*.
- A posting value $\rightarrow$ document identifier (docid).
- Number of documents in RCV1 $\rightarrow$ 800,000 documents $\rightarrow$ uses 32 bits/docid with 4-byte integers.
- Essentially we need not more than - 20 bits ($\log_2 800,000 \approx < 20$) per docid. **Why?**
- **Our goal:** use a lot less than 20 bits per docid.

Postings compression

key idea: store gaps instead of docids

- Each postings list is ordered in increasing order of docid.
 - ▶ **computer**: 283154, 283159, 283202, ...

Postings compression

key idea: store gaps instead of docids

- Each postings list is ordered in increasing order of docid.
 - ▶ **computer**: 283154, 283159, 283202, ...
- Rather than the actual docid, sufficient to store gaps:

Postings compression

key idea: store gaps instead of docids

- Each postings list is ordered in increasing order of docid.
 - ▶ **computer**: 283154, 283159, 283202, ...
- Rather than the actual docid, sufficient to store gaps:
 - ▶ $283159 - 283154 = 5$, $283202 - 283154 = 43$

Postings compression

key idea: store gaps instead of docids

- Each postings list is ordered in increasing order of docid.
 - ▶ **computer**: 283154, 283159, 283202, ...
- Rather than the actual docid, sufficient to store gaps:
 - ▶ $283159 - 283154 = 5$, $283202 - 283154 = 43$
 - ▶ **computer**: 283154, 5, 43, ...

Postings compression

key idea: store gaps instead of docids

- Each postings list is ordered in increasing order of docid.
 - ▶ **computer**: 283154, 283159, 283202, ...
- Rather than the actual docid, sufficient to store gaps:
 - ▶ $283159 - 283154 = 5$, $283202 - 283154 = 43$
 - ▶ **computer**: 283154, 5, 43, ...
- Gaps for frequent terms $\rightarrow$ small.
- Gaps for rare terms $\rightarrow$ large.

Postings compression

key idea: store gaps instead of docids

- Each postings list is ordered in increasing order of docid.
 - ▶ **computer**: 283154, 283159, 283202, ...
- Rather than the actual docid, sufficient to store gaps:
 - ▶ $283159 - 283154 = 5$, $283202 - 283154 = 43$
 - ▶ **computer**: 283154, 5, 43, ...
- Gaps for frequent terms $\rightarrow$ small.
- Gaps for rare terms $\rightarrow$ large.
- We can encode small gaps with fewer than 20 bits.

Postings compression

using gap encoding - example

	storing	postings										
the	docids	1	2	3	4	5	6		34884	34885	34886	
	gaps	1	1	1	1	1	1		1	1	0	
computer	docids	2	67	564	1567	4532	10542	27652				
	gaps	65	497	1003	2965	6010	17110	0				
holocaust	docids	652	18766	33249								
	gaps	18114	14483	0								

Postings compression

using gap encoding - example

	storing	postings									
the	docids	1	2	3	4	5	6		34884	34885	34886
	gaps	1	1	1	1	1	1		1	1	0
computer	docids	2	67	564	1567	4532	10542	27652			
	gaps	65	497	1003	2965	6010	17110	0			
holocaust	docids	652	18766	33249							
	gaps	18114	14483	0							

- Additional flexibility: can store *much more* docids than storing the docid itself.

References

- MRS - Chapter 5
- BCC - Chapter 6